

Author: Danny Castro  
Created: 28 April 2026  
Last Updated: 30 April 2026

## **CsoundUnity: A Quick Guide**

### **What is CsoundUnity**

CsoundUnity is a Unity wrapper and toolkit based off the Cabbage framework that implements most of the Csound API into Unity. In its simplest form, and without much detail, CsoundUnity works as a component that can be added to a GameObject, creating and attaching sound-based audio sources to the GameObject in question. Essentially, it gives the user the ability to run VST plugins inside of Csound, reproduce Csound instruments inside of Unity, or generate and reproduce Csound-based audio as an extension of the built-in audio system Unity offers.

*Note: It is important to mention that CsoundUnity is based off the Cabbage framework, a tool that allows us to create Csound-based DSPs. A lot of what you can do with CsoundUnity involves using Cabbage plugins. Assume that most examples mentioned in this text make use of Cabbage and not just Csound by itself (although you could).*

### **Unity Compatibility**

CsoundUnity versions 3.0 and above are compatible with Unity versions 2018.1 and above. It is compatible with Unity 6 as well, but has not yet caught up to the new Enhanced Audio Foundation and the new Scriptable Audio Pipeline, both developed in Q3 of 2025.

### **What's happening under the hood**

The wrapper populates an audiosource component with data that comes from Csound. An audiosource can be an output and an input simultaneously, which makes whatever clips are in that audiosource editable with csound (input), and then the package fills the output with whatever data was produced (e.g. how the audio was altered).

### **OnAudioFilterRead**

Csound Unity uses a specific callback called `OnAudioFilterRead`. This callback contains an array of floats that consists of audio data, such as the number of channels. The number of channels that are passed to this callback is stored as an int variable. Basically, once this callback is implemented, Unity inserts custom filters into the audio DSP chain.

OnAudioFilterRead is called every time a chunk of audio is sent to the “filter”, AKA the effect (remember that Unity calls all audio effects filters). The array goes from -1.0f to

1.0f, and can contain audio from a previous filter/effect in the DSP chain, or can also contain the audio clip assigned to the audiosource. If it's the first filter (meaning its not storing anything yet) and theres no clip assigned to the AudioSource, the filter/effect will be treated and played as the audio source. The filter/effect can be used as an audio clip that procedurally generates audio.

If there is more than one channel, then the channel data is interleaved (combined, made into a single file as an alternating sequence like LRLRLR. L and R aren't stored separately; they're combined into a single thing). Each consecutive data sample in the array comes from a different channel. Once you run out of channels, it loops back to the first one. You can find the number of samples per channel by dividing ``data.Length`` by ``channels``. (`data.Length` is used to find the size of the data)

Additionally, a VU Meter showing the outgoing sample levels will be displayed in the inspector if ``OnAudioFilterRead`` is implemented. Next to the meter, you will find the measured process time of the effect/filter in milliseconds. What this shows specifically is the process time already spent by whatever effects we have going on. If the number is red, that means the effect/filter is taking up too much time, and then we will experience audio starvation (the mixer will be starved of audio data).

**Key Features, as described by the Csound Unity GitHub repository, authored by Rory Walsh ([link here](#))**

- Support for real-time Csound rendering in Unity Multi platform support: Windows, macOS, Android, iOS
- Lots of ready-to-use examples
- Several utility methods to make your audio development easy
- An easy way of using the Csound API with C#
- An advanced Unity inspector, allowing for easy parameterization of Csound instruments, using Csound Control channels
- Support for Cabbage widgets, that are converted into Editor controls
- A simplified way of handling Csound Environment folders
- A csd editor for creating and modifying Csound code within the Unity editor
- A test score section, to be able to send a score from the CsoundUnity inspector to test your sounds
- Support for Csound Audio channels, to be able to route multichannel audio from a CsoundUnity instance to other children instances (aka CsoundUnityChild)
- A Preset system, to easily save and retrieve Control channel settings and more, also at runtime
- Support for Cabbage snaps that can be imported and converted into CsoundUnityPresets

- A FileWatcher, that detects changes on the used csd files made with an external application, and applies the changes to every CsoundUnity instance that uses them

## How to use CsoundUnity

### Setup

First, you must import the CsoundUnity wrapper into Unity as a package. There's 2 ways of doing it, and both involve using Unity's Package Manager.

1. Add the package using the option *Add package from git URL* and paste this URL: <https://github.com/rorywalsh/CsoundUnity.git>
2. Download the source code of the Latest Release (click [here](#) to access it) onto your local drive and extract its contents. Find the extracted package inside Unity's Package Manager browser and select the package.json file inside the folder. This package.json file is the one you want to add/import into Unity.

### The CsoundUnity Component

The CsoundUnity package manifests as a component that can be added to any GameObject in a scene. Create an empty GameObject (really, any GameObject will work), click on Add Component in the Inspector window, and search for the component named "CsoundUnity". Both an Audio source component and a CsoundUnity component will be added to the GameObject.

Inside the CsoundUnity component, you need to load a .csd file from Unity's asset folder. You will need to import your .csd files into Unity for this to work.

*Note: A .csd file is a Csound File, like how a .jpeg file is an image*

Once this is all done, you can run the game. **Make sure that Csound is also running that .csd file.** What will happen now is that Csound will communicate with Unity and send audio from Csound into the Audio Source component, granting Csound access to Unity's built-in audio system and its features (mixer, effects, spatialisation, etc). Csound can either be controlled from the Unity Scripts or from the Unity Editor.

### Interacting with Csound from the Unity Editor

Most, if not all, of the interactions will occur in the Inspector window belonging to a GameObject with the CsoundUnity component. The interactions make use of Cabbage's widgets (GUI elements), such as buttons, horizontal sliders, checkboxes, and comboboxes. All of the aforementioned are created inside the .csd file, and what can be altered in the Unity Inspector window are the established ranges of specific parameters (e.g. the depth and rate of an LFO).

In order for these widgets to work inside Unity's inspector (and in general), they should contain AT LEAST a channel identifier. For actual functionality, they should have a range identifier, a text identifier, and a caption identifier.

These widgets will appear in the inspector window with the default Unity skin, meaning they will look like traditional inspector controls.

*Note: CsoundUnity only supports the following basic widgets:*

- *rslider|hslider|vslider: horizontal sliders*
- *button: button with text*
- *checkbox: toggle box*
- *combobox: dropdown menu (also used for presets)*

### Interacting with Csound From a Unity C# Script

If you want to interact with Csound Unity from a C# script, you must declare a Csound Unity object like so:

```
`CsoundUnity csoundUnity;`
```

After that, you need to get the component in the start method. This will instantiate the CsoundUnity component.

```
`csoundUnity = GetComponent<CsoundUnity>()`
```

Once you have access to Csound from the C # script, you will have access to Csound instruments, ftables, and certain opcodes. Some things we can do from a Unity script are update the Csound channels, call on specific instruments, alter values and ranges, and edit presets (yes! CsoundUnity supports presets!), among others.

Make sure to check out the [Csound Unity Class References](#) for a more extensive list of what the newly integrated Csound API allows you to do in a Unity script.

### **Other CsoundUnity Components**

#### CSoundUnityChild

This component was added in version 3.0 of CsoundUnity. It lets you read the audio channels in a CsoundUnity instance, meaning there is no channel number limitation. To my understanding, it is able to read channels from external GameObjects as long as the desired CsoundUnity component is assigned in the inspector.

## CSoundFileWatcher

This component can detect changes made to the .csd file while it is being modified in an external editor and notify the user via a message displayed in the Unity Console. What this means is that alterations made to the .csd files will be updated inside of Unity without shifting assets and components around.

## Resources and Further Reading

Official CsoundUnity repository:

<https://github.com/rorywalsh/CsoundUnity>

Csound API Documentation:

<https://csound.com/docs/api/index.html>

CsoundUnity Documentation:

<https://rorywalsh.github.io/CsoundUnity/html/index.html>

CsoundUnity Class References:

[https://rorywalsh.github.io/CsoundUnity/html/class\\_csound\\_unity.html](https://rorywalsh.github.io/CsoundUnity/html/class_csound_unity.html)

The CsoundUnity Homepage:

<https://rorywalsh.github.io/CsoundUnity/#/>

Documentation on OnAudioFilterRead:

<https://docs.unity3d.com/ScriptReference/MonoBehaviour.OnAudioFilterRead.html>

Version compatibility discussion with Giovanni Bedetti in the [CabbageAudio.com](https://forum.cabbageaudio.com) forum:

<https://forum.cabbageaudio.com/t/csoundunity-version-compatibility-with-unity-6/5011>